



Aula 10: Classes Abstratas em Java

Programação Orientada a Objetos

Bem-vindos à Aula 10! Hoje exploraremos um dos pilares fundamentais da POO em Java: as **classes abstratas**. Aprenderemos como elas nos permitem definir contratos de comportamento, criar hierarquias de herança robustas e escrever código mais organizado, flexível e reutilizável.

POO EM

JAVA

AULA 10

NÍVEL INICIANTE

Objetivos de Aprendizagem

Ao final desta aula, você será capaz de:

01

Compreender o conceito

Entender o que é uma classe abstrata e por que ela existe em Java.

02

Dominar a sintaxe

Declarar classes e métodos abstratos corretamente com a palavra-chave `abstract`.

03

Aplicar em hierarquias

Criar hierarquias de herança utilizando classes abstratas como base.

04

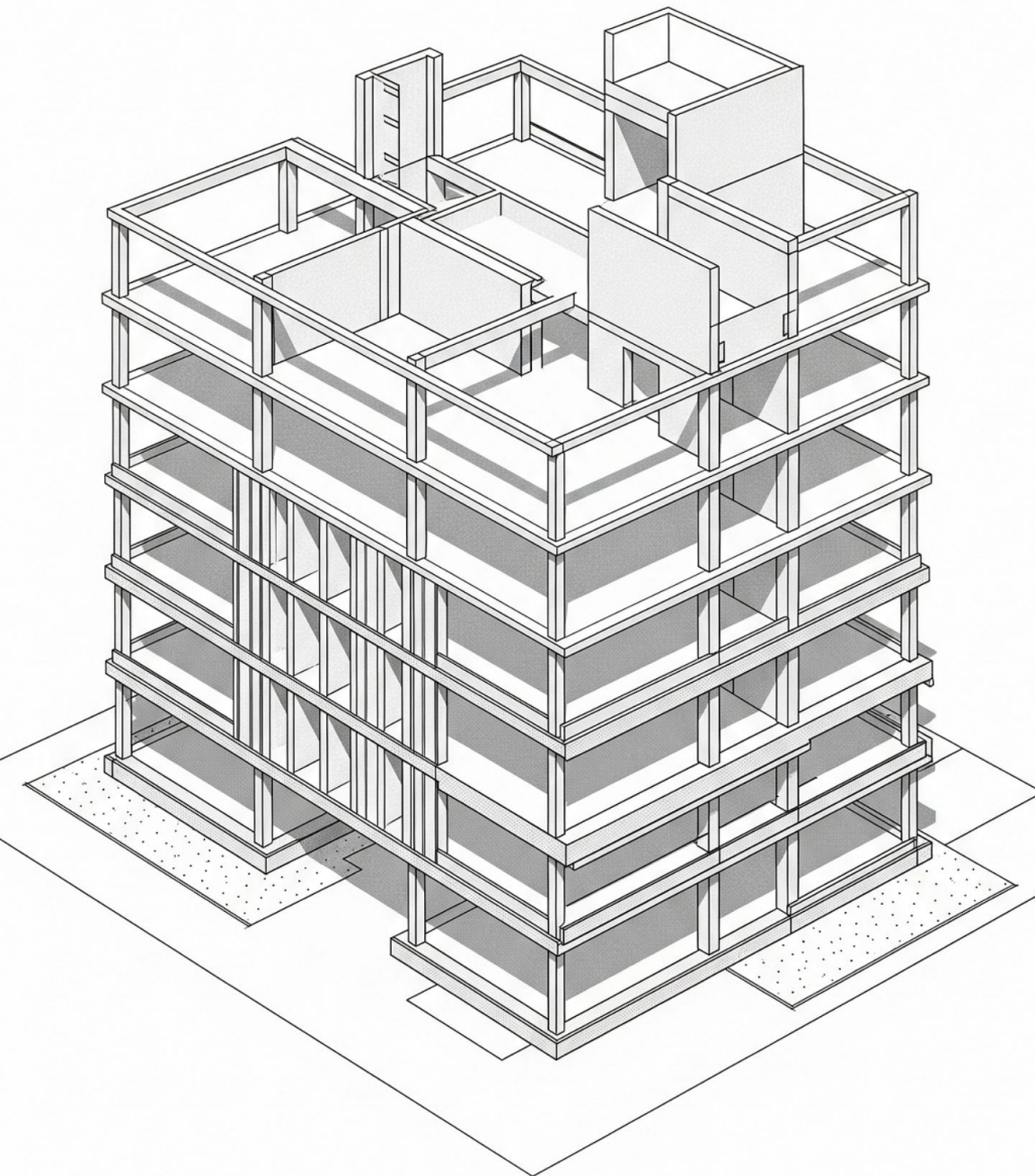
Comparar com interfaces

Identificar quando usar classe abstrata versus interface.

05

Evitar erros comuns

Reconhecer e corrigir os erros mais frequentes com classes abstratas.



O Que É uma Classe Abstrata?

Uma **classe abstrata** é uma classe que serve como **modelo** para outras classes.

Ela **não pode ser usada para criar objetos diretamente**, porque representa uma ideia mais geral ou incompleta. Em vez disso, ela é usada como base para criar subclasses, que completam ou especificam seu funcionamento.

Classe Concreta

Pode ser instanciada diretamente. Todos os métodos possuem implementação completa. Ex: `new Cachorro()`

Classe Abstrata

Não pode ser instanciada. Pode conter métodos sem implementação (abstratos). Ex: `abstract class Animal`

Características das Classes Abstratas

Não instanciável

Não é possível usar `new` para criar objetos diretamente de uma classe abstrata. Tentativas geram erro de compilação.

Métodos Abstratos e Concretos

Podem conter métodos **com** implementação (concretos) e métodos **sem** implementação (abstratos) ao mesmo tempo.

Possuem Construtores

Classes abstratas podem ter construtores, invocados pelas subclasses via `super()` durante a herança.

Possuem Atributos

Podem declarar atributos de instância, estáticos e constantes, herdados pelas subclasses normalmente.

Sintaxe em Java: Declaração

A palavra-chave `abstract` é usada tanto para declarar a classe quanto para seus métodos sem implementação. Veja a estrutura fundamental:

```
// Declaração de classe abstrata
public abstract class Animal {

    // Atributo comum a todos os animais
    private String nome;

    // Construtor (pode existir em classe
    abstrata)
    public Animal(String nome) {
        this.nome = nome;
    }
}
```

```
// Método CONCRETO: tem implementação
public String getNome() {
    return this.nome;
}

// Método ABSTRATO: sem implementação, sem
corpo {}
public abstract void emitirSom();

// Método abstrato: subclasses devem
implementar
public abstract void mover();
}
```

 **Regra importante:** Um método abstrato não possui corpo (sem `{}`). A declaração termina com ponto e vírgula.

Métodos Abstratos: Definição e Regras

O Que São?

Métodos abstratos definem uma **assinatura obrigatória** que todas as subclasses concretas devem implementar. São como cláusulas de um contrato.

```
// Método abstrato: sem corpo
public abstract double calcularArea();
public abstract void desenhar();
```

Regras Fundamentais

- Só existem em classes abstratas

Não podem ser `private`

Não podem ser `static`

Não podem ser `final`

Obrigações das Subclasses

Toda subclasse **concreta** deve implementar **todos** os métodos abstratos herdados. Caso contrário, ela mesma deve ser declarada abstrata.

```
public abstract class Forma {
    public abstract double calcularArea();
}
```

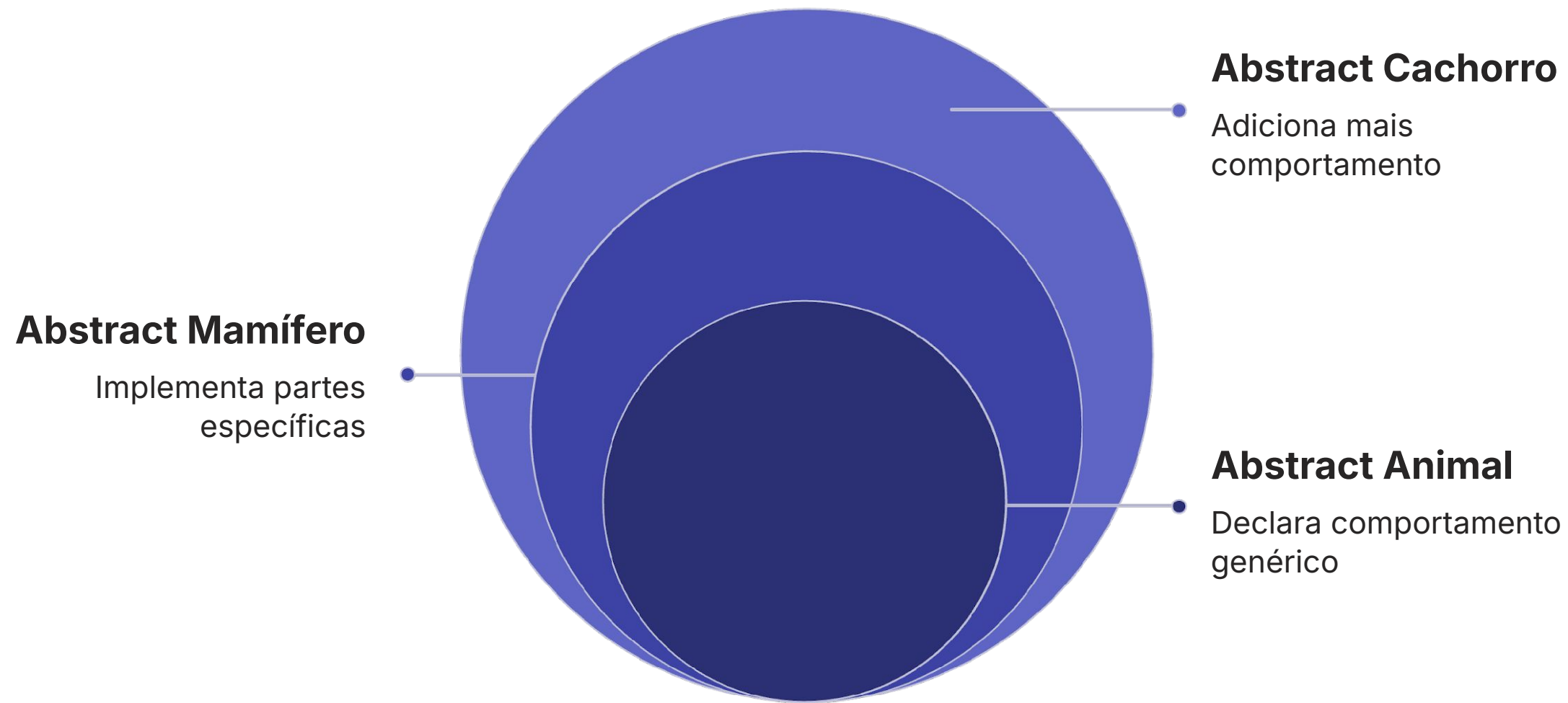
```
// Subclasse CONCRETA: implementa tudo
public class Circulo extends Forma {
    private double raio;

    @Override
    public double calcularArea() {
        return Math.PI * raio * raio;
    }
}
```

```
// Subclasse ABSTRATA: não precisa implementar
public abstract class FormaColorida
    extends Forma {
    // implementação parcial permitida
}
```

Hierarquia de Classes Abstratas

Classes abstratas podem herdar de outras classes abstratas, formando hierarquias em múltiplos níveis. Isso permite **implementação progressiva e parcial** do comportamento.



Nessa estrutura, cada nível pode adicionar métodos abstratos ou implementar parte dos herdados. Somente a classe concreta ao final da cadeia precisa implementar *todos* os métodos abstratos pendentes.

Classe Abstrata x Interface

Uma **classe abstrata** é usada quando queremos criar uma **base comum** para classes relacionadas. Ela pode possuir atributos, construtores, métodos concretos e métodos abstratos.

A subclasse herda essa estrutura usando `extends`.

```
Circulo extends Forma{  
    ...  
}
```

Ou seja: `Circulo` é uma `Forma`

Já uma **interface** é usada quando queremos definir um **contrato de comportamento**. Ela indica quais métodos uma classe deve implementar, sem obrigar que essas classes tenham a mesma origem. A classe assume esse contrato usando `implements`.

```
Circulo implements Desenhavel {...}  
Botao implements Desenhavel {...}
```

Ou seja: `Circulo` pode ser desenhado
`Botao` pode ser desenhado

❏ Use **classe abstrata** quando as classes forem parecidas e compartilharem uma base comum.

❏ Use **interface** quando classes diferentes precisarem seguir o mesmo comportamento obrigatório.

Classe Abstrata vs. Interface

Esta é uma das comparações mais importantes em POO. A escolha certa depende do relacionamento entre as classes e dos requisitos do design.

Critério	Classe Abstrata	Interface
Instanciação	Não pode ser instanciada	Não pode ser instanciada
Herança múltipla	Apenas uma superclasse	Múltiplas interfaces
Atributos de instância	Sim, permitido	Não (apenas constantes)
Construtores	Sim, permitido	Não
Modificadores de acesso	Qualquer modificador	public por padrão
Quando usar	Compartilhar código/estado	Definir contrato puro

Exemplo Prático 1: Sistema de Funcionários

Um caso clássico para classes abstratas: funcionários de uma empresa compartilham atributos comuns, mas cada tipo calcula o salário de forma diferente.

Classe Abstrata Base

```
public abstract class Funcionario {  
    private String nome;  
    private double salarioBase;  
  
    public Funcionario(String nome,  
double salarioBase) {  
        this.nome = nome;  
        this.salarioBase = salarioBase;  
    }  
  
    // Método abstrato: cada tipo  
calcula diferente  
    public abstract double  
    calcularSalario();  
}
```

```
// Método concreto: igual para  
todos  
public void exibirDados() {  
    System.out.println("Nome: " +  
nome);  
    System.out.println("Salário: R$" +  
        calcularSalario());  
}  
  
public double getSalarioBase() {  
    return salarioBase;  
}  
}
```

Exemplo Prático 1: Sistema de Funcionários

Um caso clássico para classes abstratas: funcionários de uma empresa compartilham atributos comuns, mas cada tipo calcula o salário de forma diferente.

Subclasses Concretas

```
public class Gerente extends
Funcionario {
    private double bonus;

    public Gerente(String nome, double
salarioBase, double bonus) {
        super(nome, salarioBase);
        this.bonus = bonus;
    }

    @Override
    public double calcularSalario() {
        return getSalarioBase() + bonus;
    }
}
```

```
public class Desenvolvedor extends
Funcionario {
    private int horasExtras;

    public Desenvolvedor(String nome,
double salarioBase,int horasExtras) {
        super(nome, salarioBase);
        this.horasExtras = horasExtras;
    }

    @Override
    public double calcularSalario() {
        return getSalarioBase()
            + (horasExtras * 50.0);
    }
}
```

Exemplo Prático 2: Formas Geométricas

Formas geométricas são o exemplo didático clássico de classes abstratas. Todas as formas compartilham a ideia de *área* e *perímetro*, mas cada uma os calcula de forma única.

```
public abstract class Forma {  
    private String cor;  
  
    public Forma(String cor) { this.cor = cor;  
}  
  
    // Métodos abstratos: cada forma implementa  
    public abstract double calcularArea();  
    public abstract double calcularPerimetro();
```

```
    // Método concreto compartilhado  
    public void exibir() {  
        System.out.println("Cor: " + cor  
            + " | Área: " + calcularArea()  
            + " | Perímetro: " +  
            calcularPerimetro());  
    }  
}
```

Circulo

```
double calcularArea() {  
    return Math.PI * raio * raio;  
}  
double calcularPerimetro() {  
    return 2 * Math.PI * raio;
```

Retangulo

```
double calcularArea() {  
    return largura * altura;  
}  
double calcularPerimetro() {  
    return 2*(largura + altura);}
```

Triangulo

```
double calcularArea() {  
    return (base * altura) / 2;  
}  
double calcularPerimetro() {  
    return ladoA + ladoB + ladoC;}
```

Template Method Pattern com Classes Abstratas

O **Template Method** é um padrão de projeto que utiliza classes abstratas para definir o *esqueleto de um algoritmo* na superclasse, delegando etapas específicas às subclasses. É uma das aplicações mais elegantes de classes abstratas.

Estrutura do Padrão

```
public abstract class RelatorioBase {  
  
    // Template Method: define o fluxo fixo  
    public final void gerarRelatorio() {  
        cabecalho();  
        coletarDados();    // abstrato  
        processarDados(); // abstrato  
        rodape();  
    }  
}
```

```
// Etapas concretas (fixas para todos)  
private void cabecalho() {  
    System.out.println("=== RELATÓRIO ===");  
}  
private void rodape() {  
    System.out.println("=== FIM ===");  
}  
  
// Etapas abstratas (variam por tipo)  
protected abstract void coletarDados();  
protected abstract void processarDados();  
}
```

Benefício chave: O fluxo principal (`gerarRelatorio()`) é marcado como `final`, garantindo que as subclasses não alterem a ordem das etapas — apenas personalizam cada passo.

Template *Method Pattern* com Classes Abstratas

O **Template Method** é um padrão de projeto que utiliza classes abstratas para definir o *esqueleto de um algoritmo* na superclasse, delegando etapas específicas às subclasses. É uma das aplicações mais elegantes de classes abstratas.

Subclasses Especializadas

```
public class RelatorioVendas
    extends RelatorioBase {

    @Override
    protected void coletarDados() {
        System.out.println("Coletando dados de
vendas...");
    }

    @Override
    protected void processarDados() {
        System.out.println("Calculando totais de
vendas...");
    }
}
```

```
public class RelatorioRH
    extends RelatorioBase {

    @Override
    protected void coletarDados() {
        System.out.println("Coletando dados de
funcionários...");
    }

    @Override
    protected void processarDados() {
        System.out.println("Processando folha de
pagamento...");
    }
}
```

Boas Práticas com Classes Abstratas

Design de Hierarquias Coeso

Use classes abstratas somente quando as subclasses compartilham genuinamente um relacionamento "**é um**". Um Cachorro É UM Animal. Hierarquias forçadas geram código confuso e difícil de manter.

Evitar Abstrações Excessivas

Não crie classes abstratas desnecessariamente. Se apenas uma subclasse existe, considere se a abstração é realmente necessária. A simplicidade sempre vence.

Princípio de Substituição de Liskov (LSP)

Qualquer subclasse deve poder substituir sua superclasse sem quebrar o comportamento esperado. Se uma subclasse viola contratos da classe abstrata, a hierarquia está errada.

Documentação Clara

Documente **por que** cada método é abstrato e **o que** a implementação deve fazer. Use Javadoc nas classes abstratas para guiar os implementadores corretamente.

Erros Comuns: O Que Não Fazer

Evite estes erros clássicos que causam falhas de compilação ou design ruim em Java:

✗ Instanciar classe abstrata

```
// ERRO DE COMPILAÇÃO!  
Animal a = new Animal("Rex"); // Impossível!  
// CORRETO: instanciar a subclasse concreta  
Animal a = new Cachorro("Rex");
```

✗ Não implementar métodos abstratos

```
// ERRO: Cachorro não implementou emitirSom()  
public class Cachorro extends Animal {  
    // Faltou o @Override de emitirSom()!  
}  
// SOLUÇÃO: implementar ou tornar abstrata
```

✗ Método abstrato com corpo

```
// ERRO: abstract não pode ter corpo {}  
public abstract void emitirSom() {  
    System.out.println("Som genérico");  
}  
// CORRETO: sem corpo  
public abstract void emitirSom();
```

✗ Modificadores incompatíveis

```
// ERRO: abstract não pode ser private  
private abstract void metodo();  
// ERRO: abstract não pode ser final  
public final abstract void metodo();  
// ERRO: abstract não pode ser static  
public static abstract void metodo();
```

Exemplo Completo: Animais com Comportamentos

Um exemplo integrado que demonstra herança, polimorfismo e o poder das classes abstratas em conjunto:

```
public abstract class Animal {
    private String nome;
    private int idade;

    public Animal(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    // Métodos abstratos: comportamentos únicos
    public abstract void emitirSom();
    public abstract String getTipoAlimentacao();

    // Método concreto com chamada polimórfica
    public void apresentar() {
        System.out.println("Olá! Sou " + nome
            + ", tenho " + idade + " anos.");
        System.out.println("Meu som: ");
        emitirSom(); // polimorfismo em ação!
        System.out.println("Alimentação: "
            + getTipoAlimentacao());
    }
}
```

```
public class Cachorro extends Animal {
    public Cachorro(String nome, int idade) {
        super(nome, idade);}
    @Override
    public void emitirSom() {
        System.out.println("Au au!");}
    @Override
    public String getTipoAlimentacao() {
        return "Carnívoro";}
}

// Uso polimórfico - o poder real!
public class Main {
    public static void main(String[] args) {
        Animal[] animais = {new Cachorro("Rex", 3), new
        Gato("Miau", 5), new Papagaio("Louro", 10)
        };
        for (Animal a : animais) {
            a.apresentar(); // cada um age diferente!
        }
    }
}
```

Resumo: O Que Aprendemos Hoje

abstract

Palavra-chave

Usada para declarar classes e métodos abstratos em Java

0

Instâncias diretas

Classes abstratas não podem ser instanciadas com `new`

100%

Implementação obrigatória

Subclasses concretas devem implementar todos os métodos abstratos



Classe Abstrata

Blueprint; não instanciável

Sintaxe e Regras

keyword `abstract`; sem `body`; sem `private`/`static`/`final`

Benefícios de Design

reuso, polimorfismo, Template Method

Classes abstratas são uma ferramenta poderosa para construir hierarquias de herança bem estruturadas. Elas permitem compartilhar código comum, garantir contratos de implementação nas subclasses e habilitar o polimorfismo — três pilares de um design orientado a objetos robusto e profissional.